



INDIA.RUNS

Build what next India runs on

Team Name : Sarva Automata

Team Leader Name : Arjun Mangarath

Problem Statement : Rank 100K candidates for Senior AI Engineer (search/retrieval/ranking); NDCG@10 focus; CPU-only; honeypot-aware.

Solution Overview

- Hybrid recall (BM25 + BGE dense + career RRF) → LightGBM LambdaRank → BGE cross-encoder on top-700 with tuned fusion.

Differentiation: IR + learned ranking + Redrob behavioral signals—not keywords. Explicit trap/honeypot and availability handling.

JD Understanding & Candidate Evaluation

- JD: 5–8 YOE production ML, search/retrieval/ranking, embeddings, India hubs, product over research, recruiter reachability.

Signals: silver tiers, LTR features, career evidence, GitHub, trap flags, behavioral twins (response, notice, recency). Cross-encoder for semantic fit—no runtime LLM.

Ranking Methodology

- Retrieve ~2K (BM25 + FAISS + career RRF) → LTR + penalties → CE rerank top-700 → fusion + modifiers → top-100 CSV.

Models: BM25, FAISS, LightGBM, BGE-reranker-base, Optuna-tuned fusion/modifiers.

Combine: weighted CE/LTR/RRF × behavioral multiplier – trap/clone/research penalties.

Explainability & Data Validation

- Reasoning: 5–6 templates from whitelisted fields (YOE, skills, location, availability, production vs research).

No LLM at rank time → no hallucinated justifications.

Traps: hard-exclude honeypots; clone cap top-30; availability down-rank; research penalty.

End-to-End Workflow

- Offline: preprocess → features/embeddings/BM25/FAISS → silver labels → train LTR → tune modifiers (~90 min).

Online: rank.py loads artifacts → recall → LTR → CE → fusion/modifiers → team_sarva_automata.csv + audit.

Offline: preprocess.py, train_ltr.py, tune_modifiers.py

Online: rank.py + HF Docker sandbox

Artifacts: parquet features, FAISS, BM25, ltr_model.lgb, fusion_params, modifier_params

Results & Performance

- Audit: proxy NDCG@10 = 1.0; NDCG@30 = 0.9922; tier-5 top 10/30/100 = 9/26/70; 0 honeypots in top-100; 0 low-availability in top-30.

Constraints: precomputed indexes; rank ~5–8 min CPU, 16GB RAM, no network at inference.

Technologies Used

- Python, Polars, LightGBM, sentence-transformers, FAISS, rank-bm25, FastAPI, Optuna. Gemini offline (JD + labels). Cursor for engineering assist.

Submission Assets

GitHub: github.com/ashokbugude/recruiter-candidate

Sandbox: huggingface.co/spaces/ashokbugude/redrob-ranker (upload JSONL/JSON $\leq 100 \rightarrow$ ranked CSV)

Portal CSV: [team_sarva_automata.csv](#)

Reproduce: `python rank.py --candidates ./challenge/candidates.jsonl --out ./team_sarva_automata.csv`



INDIA.RUNS

Build what next India runs on

THANK YOU